



OPEN

Object detection on low-compute edge SoCs: a reproducible benchmark and deployment guidelines

Chang Kong¹, Feng Li^{1✉}, Xiaohu Yan¹, Jinfeng Yang^{1,2}, Peng Mo³, Qiuming Luo³ & Rui Mao³

Deploying deep learning-based object detectors on low-compute edge AI SoCs remains challenging, as real-world performance depends on factors beyond nominal TOPS ratings, including architectural design, memory bandwidth, and system-level contention. This study presents a comprehensive and reproducible benchmarking of nine YOLO variants across three widely used Rockchip SoCs, covering multiple input resolutions, compute configurations, and operating conditions. Our results show that inference latency correlates more strongly with detection accuracy (mAP) than with FLOPs or parameter count, revealing the execution overhead introduced by recent architectural modules. Latency scaling with input size deviates from quadratic theoretical predictions due to bandwidth limitations, and multi-core NPU scheduling provides only marginal gains because of synchronization and shared-memory bottlenecks. Under multitasking stress, memory bandwidth emerges as the primary factor governing robustness, while energy-per-inference measurements highlight substantial efficiency differences across SoCs. These findings offer practical guidance for selecting and deploying object detection models on embedded platforms, emphasizing the need for hardware-aware model choices and memory-efficient optimizations in real-time edge AI applications.

Keywords Benchmarking, Edge AI, Object Detection, Low-compute SoCs

Edge artificial intelligence (AI) has rapidly gained importance as computer vision systems are increasingly deployed in cost-sensitive and resource-constrained environments. Among various tasks, object detection is a central component in applications such as surveillance, robotics, automotive systems, and consumer electronics^{1–4}. While cloud-based solutions can leverage high-end accelerators, practical deployment often requires running models directly on embedded devices, where low latency, limited memory bandwidth, and tight energy budgets are dominant constraints.

To address such requirements, system-on-chip (SoC) platforms that integrate CPUs, NPUs, and dedicated accelerators have become increasingly common^{5,6}. However, most benchmarking efforts have targeted high-performance edge devices—such as server-grade NVIDIA GPUs, Google TPUs, or NVIDIA Jetson platforms—rather than the low-compute SoC accelerators that dominate embedded vision applications^{7–10}. These lower-power SoCs, though ubiquitous in industrial cameras, IoT endpoints, and consumer devices, offer substantially constrained compute capacity and memory bandwidth. Their performance under realistic operating conditions, including resource contention and bandwidth saturation, remains insufficiently characterized.

In parallel, object detection models have advanced rapidly. The YOLO family, from early versions^{11–14} to more recent YOLOv5, YOLOv8, and YOLO11^{15–19}, has become one of the most widely adopted frameworks for real-time detection, offering scalable variants that balance accuracy and efficiency. Recent contributions have also emphasized lightweight adaptations of YOLO for constrained environments, including LAI-YOLOv5s for UAV applications²⁰, SOD-YOLO for small-object detection with reduced memory overhead²¹, MSGD-YOLO for low-cost edge intelligence²², FRYOLO for IoT embedded devices²³, and PCPE-YOLO with dynamically reconfigurable backbones²⁴. These studies illustrate the breadth of model-level innovations aimed at efficiency, yet how these models behave when deployed on entry-level SoCs—across different architectures, model sizes, and input resolutions—remains insufficiently studied. In particular, the interaction between algorithmic

¹Undergraduate College of Artificial Intelligence, Shenzhen Polytechnic University, Shenzhen 518055, China.

²Institute of Applied Artificial Intelligence of the Guangdong-Hong Kong Macao Greater Bay Area, Shenzhen Polytechnic University, Shenzhen 518055, China. ³College of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China. ✉email: lifeng9472@szpu.edu.cn

complexity and hardware constraints poses practical challenges for selecting the right combination of models and configurations.

To fill this gap, we present a comprehensive and reproducible benchmarking study of YOLO models on three representative Rockchip SoCs (RV1106, RK3568, RK3588). We evaluate Nano, Small, and Medium scales across multiple input resolutions, analyze compute scalability across SoCs, and examine the effects of multi-core scheduling and system-level contention. Our deployment pipeline (ONNX export, quantization, compilation, and execution) is described in detail to ensure reproducibility²⁵. This article extends our earlier conference work²⁶ by broadening the scope of evaluated SoCs and YOLO model families, incorporating detailed measurements of power consumption and energy per inference, and conducting deeper analysis of system-level constraints such as memory-bandwidth limitations, multi-core NPU scheduling, and resource contention.

Throughout this work, we use the term *latency* (or equivalently, *inference time*) to denote the end-to-end per-frame processing time measured at the NPU runtime. We define *real-time* following common practice in embedded vision: 25–30 FPS for surveillance and consumer video pipelines, and 50–60 FPS for high-speed perception tasks such as UAV obstacle avoidance. These thresholds are widely referenced in embedded robotics and vision literature.

Our quantitative analysis reveals several key observations: (i) inference latency correlates more strongly with model accuracy (mAP) than with FLOPs or parameter count; (ii) resolution scaling deviates from quadratic theoretical predictions due to memory-bandwidth limitations; (iii) performance scaling with theoretical TOPS is consistently sublinear (typically 1.3–2.0× observed speedup per 2× increase in TOPS); (iv) multi-core NPU scheduling yields only marginal gains (generally < 10%); (v) memory-bandwidth contention induces significant slowdowns on RV1106 and RK3568 (50–270% degradation), while RK3588 remains largely unaffected due to its higher sustained bandwidth; and (vi) although power draw is similar across SoCs, energy per inference varies substantially with both model scale and hardware capability, with higher-performance SoCs consistently achieving significantly higher energy efficiency. These results demonstrate the importance of hardware-aware deployment strategies and motivate further work on bandwidth- and energy-efficient model design for embedded AI.

Related work

Since the introduction of YOLOv1¹¹, the framework has undergone continuous improvements, making it one of the most widely adopted object detection architectures.¹² proposed YOLO9000, enhancing YOLOv2 with improvements from prior work, addressing the prediction of unlabeled object categories by jointly training on COCO and ImageNet, enabling real-time detection of over 9000 classes. YOLOv3¹³ enhanced detection robustness through multi-scale feature fusion, while YOLOv4¹⁴ introduced new training strategies to improve accuracy and efficiency. YOLOv5¹⁵ further optimized the architecture for scalability, and subsequent versions such as YOLOv7¹⁶ and YOLOv8 improved feature extraction and computational efficiency. YOLOv9¹⁷ and YOLOv10¹⁸ further enhance the YOLO framework by introducing advanced backbone architectures, improved feature aggregation strategies, and more efficient training techniques, leading to better accuracy-speed trade-offs and improved adaptability across diverse deployment scenarios. The latest YOLO11¹⁹ continues this trend, pushing the boundaries of detection accuracy and speed. Beyond object detection, the YOLO framework has been extended to image classification, instance segmentation, pose estimation, and oriented object detection (OBB), further broadening its applicability.

Beyond architectural advances in YOLO, a substantial body of research investigates how object detection models behave on different hardware platforms. Existing studies, however, primarily target high-compute accelerators or focus on designing new lightweight models rather than systematically benchmarking standard YOLO models on low-compute SoCs.

EdgeYOLO⁹ proposes a customized, anchor-free detector tailored for edge devices, achieving real-time performance on the NVIDIA Jetson AGX Xavier. While this work demonstrates the feasibility of designing lightweight architectures for edge inference, its focus lies in model innovation rather than in characterizing the system-level behavior of existing YOLO models across heterogeneous low-power hardware. Similarly, Zhu et al.⁸ evaluate YOLOv3 and PP-YOLO on Jetson Nano and Xavier NX, providing latency measurements and deployment recommendations for specific NVIDIA platforms. These studies offer valuable insights but are limited to GPU-based embedded devices and do not analyze the broader design space involving multiple SoCs, varying input resolutions, NPU architectures, or system resource contention.

At the datacenter scale, Jouppi et al.⁷ distill lessons learned from three generations of Google TPUs⁴ⁱ deployment, emphasizing compiler design, quantization strategies, workload evolution, and cost-effective large-scale inference. Although highly influential, this line of work focuses on industrial-grade AI accelerators rather than resource-constrained SoCs, and therefore does not address the deployment bottlenecks unique to low-bandwidth embedded NPUs.

More comprehensive benchmarking efforts such as AIBench²⁷ cover a wide range of AI tasks—including object detection—and provide benchmark suites for datacenter, HPC, IoT, and edge computing environments. However, AIBench primarily evaluates algorithmic workloads and system scalability rather than conducting fine-grained, model-level performance analyses on commercial low-compute SoCs. In particular, it does not examine how modern YOLO variants scale with input resolution, how performance is affected by memory bandwidth limitations, or how multi-core NPU scheduling and system-level contention impact real-time deployment.

In summary, prior benchmarking studies either evaluate a limited set of models on mid-to-high-end accelerators or focus on new lightweight detector designs. To the best of our knowledge, no prior work provides a systematic, reproducible, and cross-platform benchmark of contemporary YOLO models on low-compute SoCs. Furthermore, key deployment factors—such as bandwidth-driven latency scaling, sublinear TOPS-to-latency relationships, multi-core NPU scheduling overheads, energy-per-inference behavior, and real-world CPU/

SoC	CPU (cores × uArch @ GHz)	NPU (INT8 TOPS)	Memory	Bandwidth
RV1106	1 × Cortex-A7 (1.2)	0.5	256 MB DDR3L	718 MiB/s
RK3568	4 × Cortex-A55 (2.0)	1.0	2 GB LPDDR4X	2323 MiB/s
RK3588	4 × A76 (2.3) + 4 × A55 (1.8)	6.0	16 GB LPDDR4X	8807 MiB/s

Table 1. Summary of processor architectures and memory configurations for the selected Rockchip SoCs, reflecting their compute and bandwidth diversity.

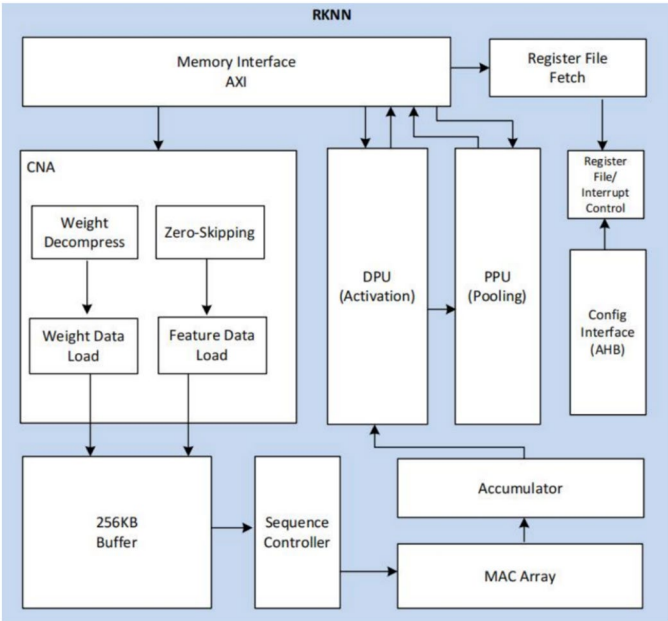


Fig. 1. The architecture of Rockchip’s third-generation NPU, which is adopted in RV1106, RK3568, and RK3588 SoCs.

memory contention—remain largely unexplored. Our work fills this gap by offering a comprehensive evaluation and practical deployment guidelines tailored for widely used Rockchip SoCs, integrating latency, bandwidth sensitivity, multi-core behavior, and power–energy measurements into a unified benchmarking study.

Methods
Hardware platforms

In order to assess the practicality of running YOLO detectors on lightweight edge processors, we investigated three Rockchip SoCs—RV1106, RK3568, and RK3588—covering diverse ranges of compute power and memory resources. These platforms are widely adopted in embedded AI applications, including smart surveillance cameras, industrial monitoring, and IoT systems. Their detailed specifications, including CPU type, NPU capacity, memory size, and measured memory bandwidth, are listed in Table 1. Notably, the bandwidth results were obtained using the mbw benchmark²⁸, reflecting practical throughput rather than theoretical peak specifications.

In addition to raw specifications, all three SoCs incorporate Rockchip’s third-generation Neural Processing Unit (NPU) architecture. The design integrates on-chip buffering, weight decompression, and zero-skipping to improve computational efficiency. As illustrated in Fig. 1, the NPU consists of the Convolution Neural Accelerator (CNA), Data Processing Unit (DPU), and Pooling Processing Unit (PPU), supported by a shared MAC array. These components form the core execution pipeline for convolutional and activation operations, while large intermediate feature maps are stored in off-chip DRAM. Similar to other embedded accelerators, practical performance on Rockchip NPUs is shaped not only by compute capacity but also by memory throughput and scheduling efficiency^{5,6}.

YOLO models

We selected three families of YOLO models—YOLOv5, YOLOv8, and YOLO11—that represent widely adopted and recently developed object detection frameworks. For each series, we examined three configurations (Nano, Small, and Medium). Their architectural characteristics, including depth, width, number of parameters, GFLOPs, and mean Average Precision (mAP) on the COCO dataset, are listed in Table 2.

Model	Depth	Width	Channels	Layers	Params	GFLOPS	mAP
YOLOv5n	0.33	0.25	1024	262	2.6	7.7	34.3
YOLOv5s	0.33	0.5	1024	262	9.1	24.0	43.0
YOLOv5m	0.67	0.75	1024	339	25.1	64.2	49.0
YOLOv8n	0.33	0.25	1024	225	3.2	8.7	37.3
YOLOv8s	0.33	0.5	1024	225	11.2	28.6	44.9
YOLOv8m	0.67	0.75	768	295	25.9	78.9	50.2
YOLO11n	0.5	0.25	1024	319	2.6	6.5	39.5
YOLO11s	0.5	0.5	1024	319	9.4	21.5	47.0
YOLO11m	0.5	1.0	512	409	20.1	68.0	51.5

Table 2. Architectural specifications of representative YOLO variants tested with 640×640 input images. Parameter counts are reported in millions (M), and mAP denotes the COCO validation score averaged over IoU thresholds 0.50–0.95.

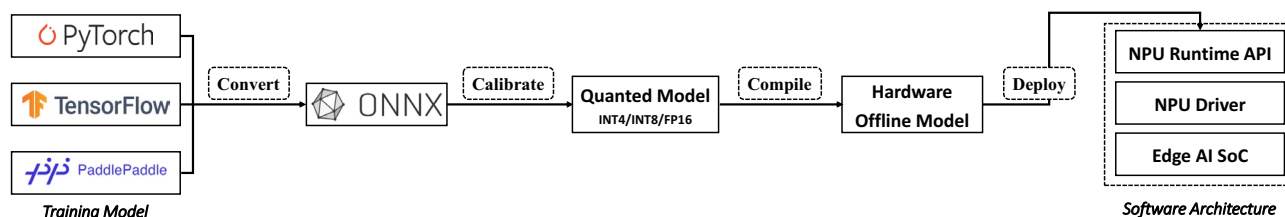


Fig. 2. Schematic diagram of inference workflow on resource-constrained AI processors.

To aid interpretation, we briefly clarify the architectural terms used throughout this paper. **C3** refers to the CSP-like bottleneck adopted in YOLOv5, composed of multiple convolutions and residual branches. **C2f**, introduced in YOLOv8, fuses intermediate features to enhance parallelism and reduce redundant operations, but requires more sophisticated backend optimizations. **C3k2**, integrated in YOLO11, modifies C3 with stacked kernels ($k = 2$), which improves representational capacity but may increase activation memory usage. In addition, the Nano, Small, and Medium variants are obtained by scaling depth and width multipliers, directly affecting parameter count, activation size, and GFLOPs. These scaling factors play an important role in determining how efficiently a model can be executed on resource-limited NPUs.

This selection covers a broad spectrum from lightweight, latency-oriented models to more accurate, compute-intensive variants. It allows us to systematically explore the trade-offs between accuracy, latency, and hardware constraints across different SoCs.

Deployment workflow

The deployment of YOLO models on Rockchip SoCs follows a multi-stage process to ensure compatibility with low-compute NPUs. The workflow is illustrated in Fig. 2, which outlines model conversion, quantization, compilation, and execution.

- **Model Conversion:** Trained YOLO models were exported into the ONNX format to ensure interoperability across platforms.
- **Quantization:** Models were quantized to INT8 using either Post-Training Quantization (PTQ) or Quantization-Aware Training (QAT)²⁵. PTQ was used for most experiments, while QAT was applied to preserve accuracy for selected Medium models.
- **Compilation:** Quantized models were compiled into Rockchip-specific offline binaries using the RKNN Toolkit, translating high-level neural operations into low-level NPU instructions.
- **Execution:** Compiled models were deployed within the Rockchip runtime environment, with inference conducted directly on the NPUs of RV1106, RK3568, and RK3588.

This workflow mirrors typical industrial deployment pipelines on embedded hardware, and ensures reproducibility by aligning with Rockchip's SDK conventions. Similar workflows are reported for Jetson and Edge TPU deployments⁸.

All models were quantized using PTQ with the RKNN Toolkit (version 2.3.0). A fixed subset of 64 images randomly sampled from the COCO 2017 validation set was used for calibration, and the same subset was applied across all YOLO variants and all SoCs to ensure consistent quantization behavior. All deployments used the original FP32 PyTorch weights; no model was retrained, fine-tuned, pruned, or structurally modified beyond quantization. We adopted the default RKNN PTQ configuration, which automatically selects symmetric or asymmetric quantization and applies channel-wise activation quantization as appropriate. The compiler's standard graph fusion and optimization passes were enabled, and no additional vendor-specific optimizations

were introduced. These unified settings ensure reproducibility and provide a fair comparison across hardware platforms.

Experimental setup and evaluation metrics

We evaluated nine YOLO variants (v5/v8/v11 in n/s/m scales) at input resolutions $\{480, 640, 800, 960, 1120, 1280\}^2$. Each model–SoC pair was tested over 100 inference runs, discarding the first 10 to eliminate initialization effects. Across all models, resolutions, and SoCs, the run-to-run latency variance remained extremely low. The standard deviation was consistently below 1% of the mean after warm-up. This stability is expected because the Rockchip NPUs operate with fixed-frequency, deterministic execution pipelines. Therefore, the mean latency is a reliable and representative metric for comparison.

Real-time feasibility was analyzed against thresholds of 25 FPS (40 ms), 30 FPS (33.3 ms), 50 FPS (20 ms), and 60 FPS (16.7 ms). System-level robustness was examined by introducing controlled CPU and memory loads using `stress-ng`²⁹.

Our evaluation emphasizes not only model complexity (parameters, FLOPs, mAP) but also SoC scalability, multi-core scheduling behavior, and sensitivity to resource contention. This comprehensive approach expands upon prior embedded benchmarking studies^{10,27}, providing detailed insights into low-compute SoC deployment.

Results

Inference latency and model complexity

To examine the relationship between model complexity and inference performance, we evaluated nine YOLO variants (YOLOv5n/s/m, YOLOv8n/s/m, YOLO11n/s/m) on the three Rockchip SoCs at a fixed input resolution of 640×640 . Latency was measured as the mean inference time over 100 runs, excluding warm-up iterations.

As shown in Fig. 3, latency varied significantly across SoCs. On the entry-level RV1106, Nano-scale models required 68–97 ms, while Medium-scale models exceeded 250 ms, making real-time inference unattainable. The mid-tier RK3568 achieved faster execution, with Nano models reduced to 34–79 ms and Medium models to 158–312 ms. The high-end RK3588, operating in single-core NPU mode, achieved the best performance: Nano variants consistently operated between 21–30 ms, sufficient for 30 FPS, while Medium variants remained below 100 ms (e.g., YOLO11m at 92 ms).

Interestingly, inference latency did not scale directly with parameter count or GFLOPs. For example, YOLO11n, with fewer parameters than YOLOv8n, exhibited higher latency on all three SoCs. Similarly, YOLOv8m consistently showed slower performance than YOLOv5m despite comparable parameter counts. Instead, latency correlated more closely with model accuracy (mAP), suggesting that architectural innovations (e.g., C2f in YOLOv8, C3k2 in YOLO11) introduce additional computational overheads that do not map efficiently onto NPUs.

These results demonstrate that, while more powerful SoCs can substantially reduce inference time, the relationship between model complexity and latency is influenced not only by FLOPs and parameters but also by architectural design choices.

Resolution scaling

We next investigated how inference latency scales with input resolution, which was varied from 480×480 to 1280×1280 across the three SoCs. Since computational complexity theoretically increases quadratically with resolution, a fourfold increase in FLOPs would be expected when doubling both height and width (e.g., from 640×640 to 1280×1280). However, measured inference latency deviated from this quadratic trend, reflecting hardware-specific bottlenecks.

As illustrated in Fig. 4, latency growth was nonlinear and strongly dependent on the SoC. On the low-power RV1106, Nano models such as YOLOv5n increased from 68 ms at 640×640 to nearly twice that at 1280×1280 , while Medium variants experienced even steeper slowdowns, far exceeding quadratic expectations. This behavior indicates that limited memory bandwidth (718 MiB/s) amplifies the effect of resolution scaling. On the mid-tier RK3568, latency grew more gradually but still displayed super-quadratic increases for larger models, with YOLO11m nearly doubling its inference time between 640×640 and 1280×1280 . By contrast, the high-end RK3588 exhibited sub-quadratic growth: Nano models increased by less than $1.5 \times$ and Medium variants by

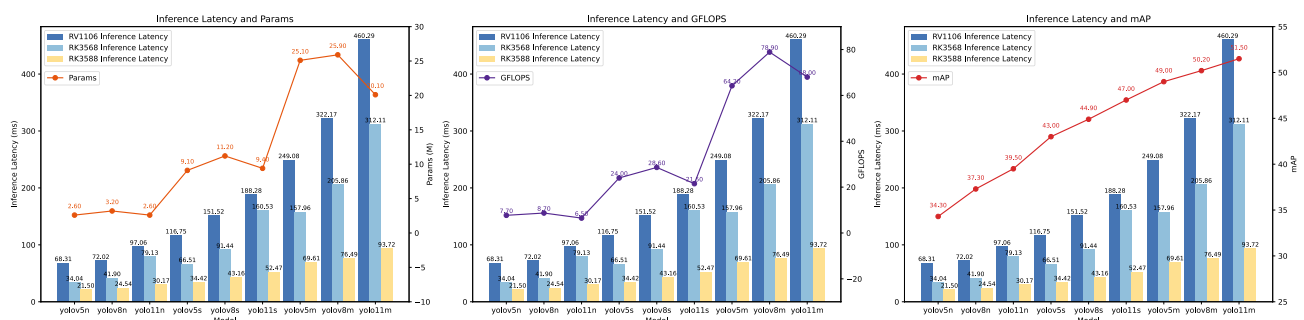


Fig. 3. Average inference time of YOLO series models evaluated on three Rockchip SoCs using 640×640 input images.

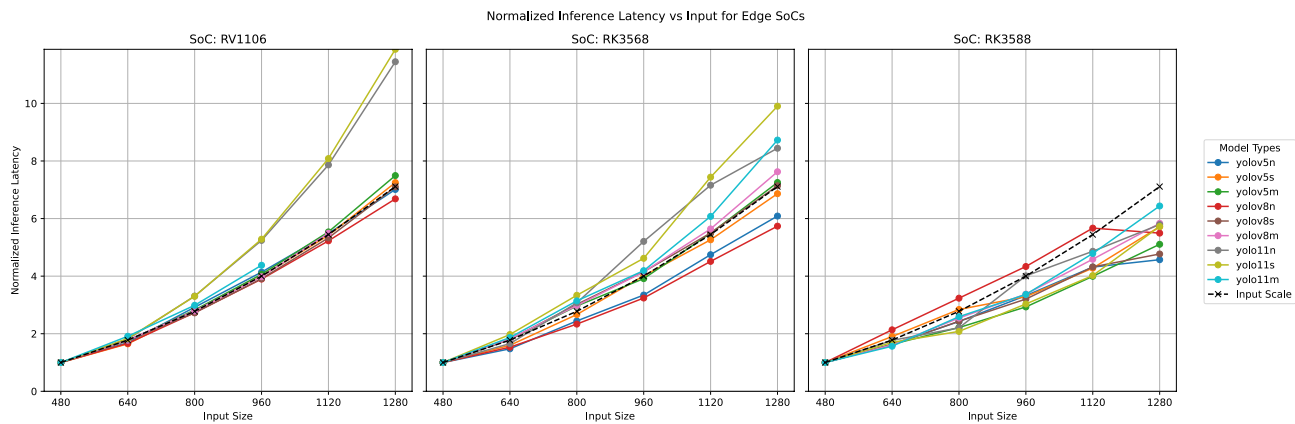


Fig. 4. Analysis of inference speed scaling with input size for YOLO-based detectors deployed on RV1106, RK3568, and RK3588 chips.

SoC	YOLOv5			YOLOv8			YOLO11		
	n	s	m	n	s	m	n	s	m
RV1106 (0.5T)	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
RK3568 (1T)	2.01	1.76	1.58	1.72	1.66	1.56	1.23	1.17	1.47
RK3588 (2T)	3.18	3.39	3.58	2.93	3.51	4.21	3.22	3.59	4.91

Table 3. Relative inference time of YOLO family detectors on Rockchip SoCs, normalized to RV1106 baseline.

approximately 1.4×, demonstrating the stabilizing effect of higher bandwidth (8807 MiB/s) and more efficient parallel processing.

Model scale further influenced sensitivity to resolution changes. Nano variants remained relatively stable across resolutions, while Medium models consistently suffered from super-quadratic growth on bandwidth-limited platforms. This highlights the disproportionate impact of memory traffic and intermediate feature map size on larger architectures.

Overall, these results show that while theoretical complexity scales quadratically with input resolution, practical inference latency can be either sub-quadratic or super-quadratic, depending on the SoC’s memory subsystem and NPU scheduling. For low-compute devices, high-resolution inputs quickly become impractical, whereas higher-end SoCs retain usable performance even at 1280×1280.

Sublinear compute scalability

We next analyzed how inference latency scales with the theoretical compute power of the three SoCs. The comparison was made between RV1106 (0.5 TOPS), RK3568 (1 TOPS), and RK3588 (2 TOPS, single-core mode), using the same set of nine YOLO models at 640×640 resolution. Latency values were normalized to RV1106 to highlight relative improvements.

As summarized in Table 3, latency consistently decreased as compute capacity increased, but the improvements were sublinear relative to the expected ratios. For example, YOLOv5n achieved a 2.0× speedup on RK3568 compared to RV1106, while YOLO11s improved by only 1.2×. On RK3588, the gains ranged from 2.9× (YOLOv8n) to 4.9× (YOLO11m), still below the theoretical 4× reduction expected when moving from 0.5 TOPS to 2 TOPS.

Scaling behavior also varied with model size. Larger models such as YOLO11m and YOLOv8m benefited more from increased compute resources, achieving speedups close to 5× on RK3588. In contrast, smaller models such as YOLOv8n and YOLO11n showed more limited improvements, indicating that factors other than raw compute power constrain performance.

Overall, these results show that inference latency does not scale proportionally with theoretical compute capacity. Differences in memory bandwidth, scheduling efficiency, and hardware utilization introduce bottlenecks that prevent SoCs from achieving ideal scaling.

Multi-core scheduling efficiency

To assess the effect of workload distribution across multiple NPU cores, we benchmarked the nine YOLO models on RK3588, which integrates a three-core NPU. Experiments were conducted under different scheduling modes, including single-core execution, dual-core, three-core, and automatic scheduling.

The results, summarized in Table 4, show that multi-core scheduling did not lead to proportional reductions in latency. For small models such as YOLOv5n, the difference between single-core and multi-core execution was less than 5% (21.5 ms vs. 20.5 ms). For larger models, the gains were similarly modest or even negligible.

Model	AUTO	C0	C1	C2	C0_1	C0_1_2	ALL
YOLOv5n	21.7	21.5	21.46	22.07	20.96	21.07	20.47
YOLOv5s	34.08	34.42	28.88	34.53	28.49	27.95	28.33
YOLOv5m	69.58	69.61	69.9	59.84	58.81	68.1	58.34
YOLOv8n	24.88	24.54	26.1	25.89	25.31	25.33	21.25
YOLOv8s	43.27	43.16	43.25	42.72	42.61	41.88	41.94
YOLOv8m	90.2	76.49	77.11	77.16	74.32	73.83	74.09
YOLO11n	30.71	30.17	31.65	30.72	31.71	31.27	30.49
YOLO11s	41.25	52.47	52.68	42.2	41.37	52.4	51.71
YOLO11m	92.22	93.72	98.05	94.11	95.47	96.01	95.51

Table 4. Inference latency (ms) of YOLO models under different core scheduling strategies on RK3588.

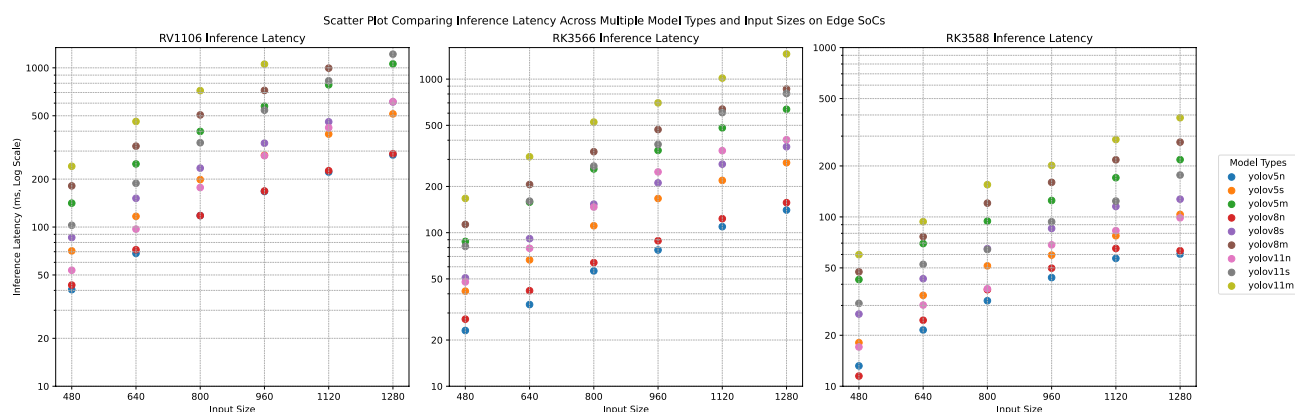


Fig. 5. Real-time feasibility analysis of YOLO series models across edge AI SoCs.

For example, YOLO11m achieved 93.7 ms in single-core mode and 95.5 ms in three-core mode, indicating that overhead from synchronization and memory contention offset the benefits of parallel execution. Automatic scheduling also failed to outperform manual configurations and, in some cases, produced slightly higher latencies.

These findings suggest that while RK3588 provides hardware support for multi-core inference, the efficiency of current scheduling mechanisms remains limited. Performance improvements are constrained by inter-core communication, synchronization overhead, and shared memory bandwidth, preventing linear scaling with the number of cores.

Real-time feasibility

To evaluate whether the tested models can meet the requirements of real-time applications, we analyzed inference latency against common frame-rate thresholds: 25 FPS (40 ms), 30 FPS (33.3 ms), 50 FPS (20 ms), and 60 FPS (16.7 ms).

As shown in Fig. 5, real-time feasibility strongly depends on both the SoC and the model scale. On the entry-level RV1106, none of the models met the 25 FPS requirement, even at lower resolutions, confirming that this platform is unsuitable for real-time object detection with current YOLO variants. On the mid-tier RK3568, Nano models and some Small variants satisfied the 25–30 FPS thresholds at 640×640 or below, but performance degraded quickly at higher resolutions. In contrast, the high-end RK3588 achieved real-time operation for all Nano and Small models, maintaining 30 FPS across most tested resolutions. However, Medium-scale models remained above 40 ms in many cases, and none of the models consistently achieved the stricter 50–60 FPS requirements.

These results demonstrate that while real-time inference is achievable on RK3588 for lightweight YOLO models, ultra-low-latency targets (50–60 FPS) remain challenging, particularly for larger architectures and higher input resolutions. On lower-end SoCs, only highly constrained use cases are feasible without further optimization.

System-level resource contention

To assess the robustness of inference under realistic multitasking scenarios, we introduced controlled CPU and memory loads while running the nine YOLO models at 640×640 resolution. CPU utilization levels were set to 30%, 50%, 70%, and 90%, while memory loads were configured at approximately 25%, 50%, and 75% of each SoC's measured bandwidth. CPU load was generated using the `cpu` class in `stress-ng`, while memory pressure was applied using the `vm` class, saturating DRAM bandwidth with streaming memory operations.

The results, summarized in Fig. 6, reveal that inference latency was more sensitive to memory contention than to CPU utilization across all platforms. On the entry-level RV1106, latency for YOLOv5n increased from 68 ms (no load) to 107 ms under high memory load, a 57% slowdown, whereas CPU stress at 90% utilization increased latency by only 13%. Similar trends were observed for larger models, with memory load consistently producing greater degradation. The mid-tier RK3568 showed even stronger sensitivity: for YOLOv5n, latency increased by more than 270% under heavy memory load, while CPU load effects remained below 30% for most

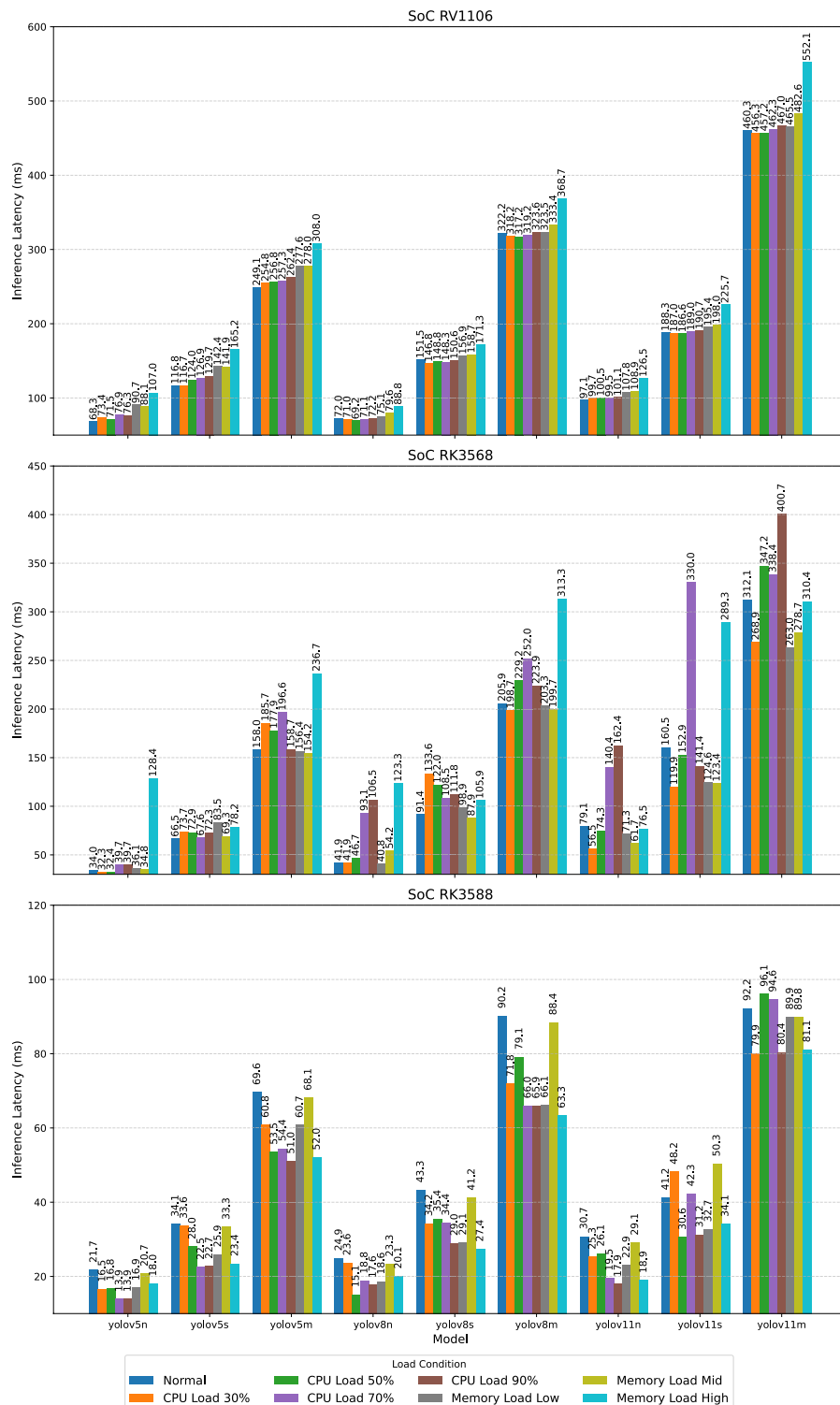


Fig. 6. Inference latency of YOLO models under varying CPU and memory loads on RV1106, RK3568, and RK3588 SoCs at 640×640 resolution.

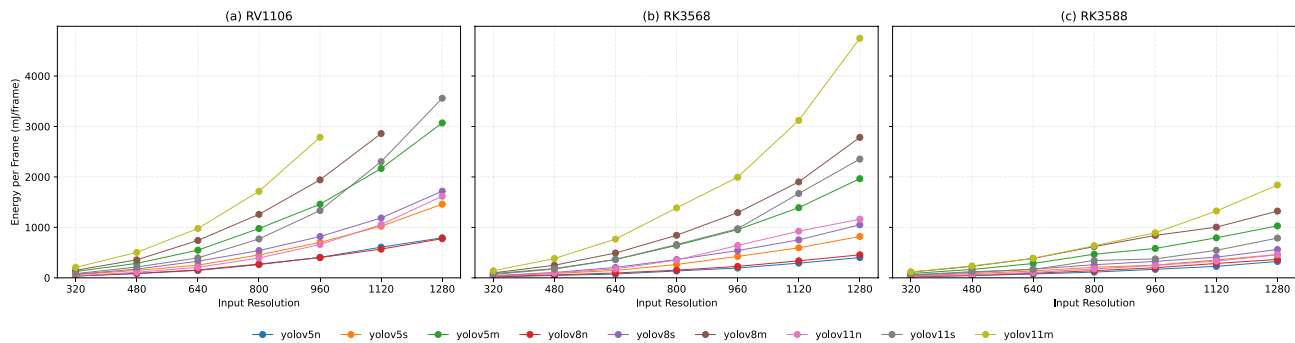


Fig. 7. Energy per inference across input resolutions on the three Rockchip SoCs. (a) RV1106, (b) RK3568, and (c) RK3588. The y-axis shows the energy per frame in millijoules (mJ/frame).

	RV1106		RK3568		RK3588	
Model	Power [W]	Energy [mJ/frame]	Power [W]	Energy [mJ/frame]	Power [W]	Energy [mJ/frame]
yolov5n	2.16	147.23	2.22	75.52	3.45	74.93
yolov5s	2.15	251.08	2.21	146.70	4.00	136.42
yolov5m	2.20	548.44	2.30	363.55	4.08	284.20
yolov8n	2.18	156.90	2.29	96.03	3.68	91.65
yolov8s	2.17	329.39	2.26	206.36	3.97	171.59
yolov8m	2.30	740.90	2.39	492.19	4.27	385.44
yolov11n	2.15	208.28	2.24	177.57	3.51	107.69
yolov11s	2.10	395.26	2.27	364.85	4.04	166.66
yolov11m	2.12	978.10	2.46	768.14	4.18	385.48

Table 5. Average power and energy per inference at 640×640 input resolution for all evaluated YOLO models on the three Rockchip SoCs.

cases. By contrast, the high-end RK3588 demonstrated resilience, with latency changes remaining below 5% even under heavy memory or CPU stress.

These findings highlight memory bandwidth as the dominant factor affecting inference robustness in resource-constrained SoCs. While CPU utilization introduces some variability, the ability of NPUs to offload compute limits its impact. In contrast, memory-intensive operations—especially for larger models—compete directly with other system tasks, leading to significant slowdowns on devices with limited bandwidth.

Power consumption and energy efficiency

Power consumption and energy-per-inference are critical metrics for evaluating the practicality of object detection models on low-compute edge SoCs, especially in embedded and battery-powered deployments. To characterize the energy efficiency of different YOLO variants across platforms, we measured the average power draw and computed the per-frame energy consumption for RV1106, RK3568, and RK3588 under continuous inference. Power was recorded using an external USB power meter under steady-state conditions. Energy per inference (mJ/frame) was obtained as: $E = P \times t$, where P is the average power consumption (W) and t is the measured per-frame latency (s). This formulation directly reflects the effective energy cost of running a single model inference on each SoC.

Figure 7 illustrates the energy-per-frame scaling trend across resolutions from 320 to 1280. All three SoCs exhibit monotonic growth in energy consumption with increasing input resolution; however, the rate of growth differs significantly across platforms. RV1106 shows the steepest scaling behavior, with energy increasing from tens of millijoules at low resolutions to several joules for medium-sized models at 1280×1280. RK3568 demonstrates a more moderate trend, benefiting from higher compute density and improved memory bandwidth. RK3588 achieves the best energy scaling: even at high resolutions, its energy-per-frame remains substantially lower due to its significantly reduced latency.

The differences across model families are also evident. Nano models maintain low energy consumption across all SoCs, ranging from roughly 20–80 mJ/frame on RK3588 and RK3568, and 100–150 mJ/frame on RV1106. In contrast, medium-scale models impose a much higher energy cost: YOLOv11m reaches 978 mJ/frame on RV1106, 768 mJ/frame on RK3568, and 385 mJ/frame on RK3588 at the 640×640 resolution. These differences underscore the importance of selecting appropriate model sizes for constrained hardware.

Table 5 summarizes the detailed measurements at the commonly used input resolution of 640×640. RV1106 generally operates around 2.1–2.3 W across all models, RK3568 ranges from 2.2–2.5 W, while RK3588 consumes between 3.4–4.2 W. Despite drawing more power, RK3588 is consistently the most energy-efficient device due

to its significantly lower latency, leading to up to $3\times$ lower energy-per-frame compared to RV1106 for medium-sized models. This illustrates a key conclusion: higher power consumption does not necessarily imply higher energy cost, and higher-performance SoCs may achieve superior efficiency per inference.

Overall, the power and energy measurements highlight a clear design guideline: (1). RV1106 is suitable only for nano/small models at moderate resolutions. (2). RK3568 supports nano and small models efficiently and can run medium models when energy constraints are relaxed. (3). RK3588 offers the best energy-per-inference performance, enabling deployment of medium-scale YOLO variants while preserving favorable energy efficiency. These results provide a practical reference for selecting model–hardware combinations in resource-constrained embedded or battery-powered applications.

Discussion

Model-level insights: architecture vs. complexity

The results reveal that inference performance on low-compute SoCs is shaped more profoundly by architectural design choices than by conventional model complexity metrics such as parameter count or FLOPs. Across all platforms, models with higher mAP—typically incorporating more expressive modules such as C2f (YOLOv8) or C3k2 (YOLO11)—exhibited noticeably higher latency despite having similar or even smaller FLOP counts compared to earlier YOLO variants. These modules improve representational capacity but introduce additional feature-map transformations that do not map efficiently onto Rockchip NPUs. The strong correlation between latency and accuracy (mAP), and the weak correlation with FLOPs, indicate that FLOPs alone are insufficient predictors of real-world inference costs. This highlights the importance of benchmarking model variants directly on the target SoC rather than relying on theoretical indicators or GPU-based measurements.

Resolution scaling and memory-bandwidth interactions

Scaling the input resolution exposed fundamental limits imposed by memory subsystems. Although theoretical inference cost should grow quadratically with resolution, empirical scaling deviated substantially across SoCs due to bandwidth constraints. On RV1106 and RK3568, latency increased super-quadratically for Medium-scale models, driven by the sharp rise in intermediate feature-map sizes and associated DRAM traffic. These devices possess measured DRAM bandwidths of only 718 MiB/s and 2323 MiB/s, respectively, making them highly susceptible to bandwidth saturation at higher resolutions. In contrast, RK3588—with 8807 MiB/s bandwidth and more efficient parallelism—exhibited sub-quadratic scaling across all YOLO variants. The stress-ng experiments further support these findings: memory pressure inflates latency by 50–270% on RV1106 and RK3568, mirroring the effects observed with large input resolutions. These results show that resolution scaling is fundamentally bounded by available memory throughput rather than compute alone, making high-resolution inference feasible only on higher-end SoCs. As shown in Fig. 1, the Rockchip NPU relies on off-chip DRAM for large feature maps, which explains the bandwidth-sensitive behavior observed in our experiments.

Compute scalability and NPU scheduling efficiency

Comparative analysis across SoCs demonstrated a consistent gap between theoretical compute capacity (TOPS) and achievable speedups. While transitioning from RV1106 (0.5 TOPS) to RK3568 (1 TOPS) or RK3588 (2 TOPS in single-core mode) improved latency, the gains were consistently sublinear. Smaller models, which are more memory-bound, benefited the least, while Medium-scale models achieved greater speedups due to more compute-bound behavior. Furthermore, multi-core NPU scheduling on RK3588 did not yield proportional latency reductions. For most models, three-core configurations offered marginal or even negative improvements compared to single-core mode. This is attributable to inter-core synchronization overhead and shared-memory contention, which dominate the execution pipeline when multiple cores attempt to access overlapping regions of activation memory. These findings imply that both compute and memory subsystems must be co-optimized for effective multi-core utilization; simply increasing available TOPS does not guarantee proportional latency reductions.

System robustness and energy efficiency

System-level stress experiments revealed a substantial disparity in robustness across SoCs. CPU utilization had relatively minor impact on inference latency, as the compute-intensive portion of YOLO execution is handled by the NPU. However, memory contention produced substantial slowdowns on RV1106 and RK3568, reaching 50–270% degradation depending on model scale. RK3588, with its significantly higher bandwidth, maintained latency stability within 5% even under heavy memory pressure, underscoring bandwidth as the dominant constraint in low-compute environments.

Energy-per-inference analysis further highlights the practical differences among SoCs. Although power consumption remained relatively stable within each platform, the resulting energy-per-frame varied drastically due to latency differences. For Nano models at 640×640 , energy ranged from 20–80 mJ/frame on RK3588 and RK3566, compared to 100–150 mJ/frame on RV1106. Medium-scale models consumed 300–1000 mJ/frame, with RK3588 achieving $2\text{--}3\times$ greater energy efficiency than RV1106. These findings emphasize that higher absolute power draw does not necessarily imply higher energy cost, and that high-performance SoCs may be more suitable for battery-powered systems when per-inference energy is the primary constraint.

Deployment implications and practical guidelines

The combined results have direct implications for real-world embedded applications. In surveillance scenarios operating at 25–30 FPS with 720p video, only Nano and Small models on RK3588 meet real-time constraints, while RK3568 supports limited configurations and RV1106 falls short across all tested models. UAVs and mobile robots require end-to-end latencies of 20–40 ms, which restrict feasible deployments to Nano variants

on RK3588. IoT devices with relaxed FPS requirements may adopt Nano or Small variants on RK3568, while RV1106 is limited to low-resolution, low-FPS settings.

The results also suggest several software optimization strategies to improve deployment efficiency on constrained SoCs. Operator fusion and layer reordering can reduce intermediate memory traffic; activation compression or lower-precision intermediate tensors can alleviate bandwidth stress; and memory-aware scheduling may improve multi-core utilization on future NPU architectures. Together, these observations motivate hardware–software co-design for optimizing inference on bandwidth-limited, low-compute edge processors.

Overall, this study provides a practical framework for selecting and deploying YOLO models on resource-constrained SoCs. The findings highlight the interplay between model architecture, resolution, compute capacity, memory bandwidth, and system-level conditions, and underline the importance of hardware-aware deployment strategies in edge AI applications.

Conclusion

This work presents the first comprehensive and reproducible benchmarking of contemporary YOLO object detectors on low-compute Rockchip SoCs, evaluating nine model variants across multiple resolutions, bandwidth conditions, and power regimes. The results demonstrate that inference performance on embedded NPUs is shaped jointly by model architecture, memory-bandwidth constraints, and system-level behavior rather than by theoretical compute (TOPS) or FLOPs alone.

Three overarching conclusions emerge from our analysis. First, inference latency correlates more strongly with mAP and architectural complexity than with parameter count or FLOPs, indicating that modern modules such as C2f and C3k2 impose additional overhead on constrained NPUs. Second, resolution scaling and stress-induced slowdowns reveal memory bandwidth as the dominant limiting factor on RV1106 and RK3568, while RK3588's higher throughput enables sub-quadratic scaling and robust performance under contention. Third, both compute scalability and multi-core NPU scheduling exhibit sublinear behavior, highlighting that synchronization and shared-memory interactions hinder parallel efficiency. Energy-per-inference measurements further reinforce these trends: although power draw varies modestly across SoCs, the resulting energy cost per frame differs by up to $3\times$ due to latency disparities.

These findings underscore the importance of hardware-aware deployment strategies. Lightweight models and moderate resolutions are best suited for low-compute SoCs, whereas high-end platforms can support larger architectures with improved latency and energy efficiency. Memory-centric optimizations—such as operator fusion, activation compression, and bandwidth-aware scheduling—represent promising directions for further improving performance on embedded NPUs.

By providing a unified evaluation methodology, detailed measurements, and cross-platform comparisons, this work offers a practical reference framework for deploying object detection models in real-world edge scenarios. The insights presented here directly inform applications in smart surveillance, robotics, and IoT, where achieving the right balance of accuracy, latency, and energy efficiency is essential for reliable and scalable system design.

Data availability

The data that support the findings of this study are available from the corresponding author, Dr. Feng Li, upon reasonable request.

Received: 1 October 2025; Accepted: 16 January 2026

Published online: 21 January 2026

References

- Nayak, R., Pati, U. C. & Das, S. K. A comprehensive review on deep learning-based methods for video anomaly detection. *Image and Vision Computing* **106**, 104078, <https://doi.org/10.1016/j.imavis.2020.104078> (2021).
- Lu, L. et al. Deep learning-assisted real-time defect detection and closed-loop adjustment for additive manufacturing of continuous fiber-reinforced polymer composites. *Robotics and Computer-Integrated Manufacturing* **79**, 102431. <https://doi.org/10.1016/j.rcim.2022.102431> (2023).
- Berens, F. et al. Adaptive training for robust object detection in autonomous driving environments. *IEEE Transactions on Intelligent Vehicles* <https://doi.org/10.1109/TIV.2024.3439001> (2024).
- Ahmed, F. A. et al. Deep learning for surgical instrument recognition and segmentation in robotic-assisted surgeries: A systematic review. *Artificial Intelligence Review* **58**, 1. <https://doi.org/10.1007/s10462-024-10979-w> (2024).
- Mohaidat, T. & Khalil, K. A survey on neural network hardware accelerators. *IEEE Transactions on Artificial Intelligence* **5**, 3801–3822. <https://doi.org/10.1109/TAI.2024.3377147> (2024).
- Shuvo, M. M. H., Islam, S. K., Cheng, J. & Morshed, B. I. Efficient acceleration of deep learning inference on resource-constrained edge devices: A review. *Proceedings of the IEEE* **111**, 42–91. <https://doi.org/10.1109/JPROC.2022.3226481> (2023).
- Jouppi, N. P., Yoon, D. H., Ashcraft, M., Gottscho, M., Jablin, T. B., Kurian, G., Laudon, J., Li, S., Ma, P., Ma, X. and Norrie, T. (2021) Ten lessons from three generations shaped google's tpuv4i: Industrial product. *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, <https://doi.org/10.1109/ISCA52012.2021.00010>.
- Zhu, J., Feng, H., Zhong, S. & Yuan, T. Performance analysis of real-time object detection on jetson device. In *2022 IEEE/ACIS 22nd International Conference on Computer and Information Science (ICIS)*, 156–161, <https://doi.org/10.1109/ICIS54925.2022.9882480> (2022).
- Liu, S., Zha, J., Sun, J., Li, Z. & Wang, G. Edgelyolo: An edge-real-time object detector. In *2023 42nd Chinese Control Conference (CCC)*, 7507–7512, <https://doi.org/10.23919/CCC58697.2023.10239786> (2023).
- Lazarevich, I. et al. Yolobench: Benchmarking efficient object detectors on embedded systems. In *2023 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 1161–1170, <https://doi.org/10.1109/ICCVW60793.2023.00126> (2023).
- Redmon, J., Divvala, S., Girshick, R. & Farhadi, A. You only look once: Unified, real-time object detection, <https://doi.org/10.48550/arXiv.1506.02640> (2016). [arXiv:1506.02640](https://arxiv.org/abs/1506.02640).

12. Redmon, J. & Farhadi, A. Yolo9000: Better, faster, stronger. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 6517–6525, <https://doi.org/10.1109/CVPR.2017.690> (2017).
13. Redmon, J. & Farhadi, A. Yolov3: An incremental improvement, <https://doi.org/10.48550/arXiv.1804.02767> (2018). [arXiv:1804.02767](https://doi.org/10.48550/arXiv.1804.02767).
14. Bochkovskiy, A., Wang, C.-Y. & Liao, H.-Y. M. Yolov4: Optimal speed and accuracy of object detection, <https://doi.org/10.48550/arXiv.2004.10934> (2020). [arXiv:2004.10934](https://doi.org/10.48550/arXiv.2004.10934).
15. Jocher, G. Yolov5 by ultralytics, <https://doi.org/10.5281/zenodo.3908559> (2020).
16. Wang, C.-Y., Bochkovskiy, A. & Liao, H.-Y. M. Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 7464–7475, <https://doi.org/10.1109/CVPR52729.2023.00721> (2023).
17. Wang, C.-Y., Yeh, I.-H. & Mark Liao, H.-Y. Yolov9: Learning what you want to learn using programmable gradient information. In Leonardis, A. et al. (eds.) *Computer Vision – ECCV 2024*, 1–21, https://doi.org/10.1007/978-3-031-72751-1_1 (Springer Nature Switzerland, 2024).
18. Wang, A. et al. Yolov10: Real-time end-to-end object detection, <https://doi.org/10.48550/arXiv.2405.14458> (2024). [arXiv:2405.14458](https://doi.org/10.48550/arXiv.2405.14458).
19. Jocher, G., Qiu, J. & Chaurasia, A. Ultralytics yolo (2023).
20. Deng, L. et al. Lightweight aerial image object detection algorithm based on improved YOLOv5s. *Scientific Reports* **13**, 7817. <https://doi.org/10.1038/s41598-023-34892-4> (2023).
21. Xiao, Y. & Di, N. SOD-YOLO: a lightweight small object detection framework. *Scientific Reports* **14**, 25624. <https://doi.org/10.1038/s41598-024-77513-4> (2024).
22. Xiao, L., Li, W., Yao, S., Liu, H. & Ren, D. High-precision and lightweight small-target detection algorithm for low-cost edge intelligence. *Scientific Reports* **14**, 23542. <https://doi.org/10.1038/s41598-024-75243-1> (2024).
23. Chen, R. et al. An optimized lightweight real-time detection network model for IoT embedded devices. *Scientific Reports* **15**, 3839. <https://doi.org/10.1038/s41598-025-88439-w> (2025).
24. Chen, W., Liu, J., Liu, T. & Zhuang, Y. PCPE-YOLO with a lightweight dynamically reconfigurable backbone for small object detection. *Scientific Reports* **15**, 29988. <https://doi.org/10.1038/s41598-025-15975-w> (2025).
25. Jacob, B. et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2704–2713, <https://doi.org/10.1109/CVPR.2018.00286> (2018).
26. Kong, C., Mo, P., Zeng, T., Luo, Q. & Mao, R. Challenges and benchmarking of object detection models on edge AI SoCs. In Huang, D.-S., Chen, H., Li, B. & Zhang, Q. (eds.) *Advanced Intelligent Computing Technology and Applications*, 253–264, https://doi.org/10.1007/978-981-96-9908-7_21 (Springer Nature, Singapore, 2025).
27. Gao, W. et al. Aibench: Towards scalable and comprehensive datacenter ai benchmarking. In Zheng, C. & Zhan, J. (eds.) *Benchmarking, Measuring, and Optimizing*, 3–9, https://doi.org/10.1007/978-3-030-32813-9_1 (Springer International Publishing, 2019).
28. Horvath, A., j.m.slocum & Willian.Zhang. mbw: Memory bandwidth benchmark. <https://github.com/raas/mbw> (2003). Version: Latest commit c3155b5.
29. King, C. I. stress-ng: A tool to load and stress a computer system. <https://github.com/ColinIanKing/stress-ng> (2013). Version: v0.18.10.

Acknowledgements

This work was supported by the Post-doctoral Later-stage Foundation Project of Shenzhen Polytechnic University (Grant No. 6023271039K), the Research Projects of the Department of Education of Guangdong Province (Grant No. 2024KTSCX052), and the Shenzhen Polytechnic University Research Fund (Grant Nos. 6023310030K and 6024310045K).

Author contributions

C.K. conceived the study, designed the experiments, and wrote the initial draft of the manuscript. F.L. supervised the project, provided critical revisions, and approved the final version of the paper. X.Y. and J.Y. performed experiments and data collection. P.M. and Q.L. contributed to model deployment and hardware benchmarking. R.M. assisted with data analysis and visualization. All authors discussed the results and contributed to the final manuscript.

Funding

The Post-doctoral Later-stage Foundation Project of Shenzhen Polytechnic University (Grant No. 6023271039K), the Research Projects of the Department of Education of Guangdong Province (Grant No. 2024KTSCX052), and the Shenzhen Polytechnic University Research Fund (Grant Nos. 6023310030K and 6024310045K).

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to F.L.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026